Python Competitive Programming Reference

Compact team reference — algorithms, mathematics, graphs, and data structures

Python Syntax, Formatting & Fast I/O

```
# Binary string -> integer
n = int(s, 2)

# Integer -> zero-padded binary string
b = format(x, f"0{width}b")

# Float formatting
print("%.2f" % value)
print(f"{answer:.2f}")

# Read entire input as integers
import sys
data = list(map(int, sys.stdin.buffer.read().split()))

# Read entire input as strings
words = sys.stdin.buffer.read().split()
# bytes -> str only when needed: token.decode()
```

Sorting a defaultdict

```
from collections import defaultdict

# By key
D = defaultdict(d.default_factory,
                sorted(d.items()))

# By value
D = defaultdict(d.default_factory,
                sorted(d.items(),
                       key=lambda kv: kv[1]))
```

Modular Arithmetic

For modulus M:

$$(a+b) \bmod M = ((a \bmod M) + (b \bmod M)) \bmod M$$

$$(a-b) \bmod M = ((a \bmod M) - (b \bmod M) + M) \bmod M$$

$$(ab) \bmod M = ((a \bmod M)(b \bmod M)) \bmod M$$

In Python, `a % M` is already non-negative for positive M.

Sequences & Series

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

$$\sum_{k=0}^{n-1} (a+kd) = \frac{n(2a+(n-1)d)}{2}$$

$$\sum_{i=1}^{n} (2i-1) = n^2$$

$$\sum_{i=1}^{n} 2i = n(n+1)$$

Arithmetic progression squares:

$$\sum_{k=0}^{n-1} (a+kd)^2 = na^2 + ad\,n(n-1) + d^2\frac{n(n-1)(2n-1)}{6}$$

Arithmetic progression cubes:

$$\sum_{k=0}^{n-1} (a+kd)^3 = na^3 + 3a^2d\frac{n(n-1)}{2} + 3ad^2\frac{n(n-1)(2n-1)}{6} + d^3\left(\frac{n(n-1)}{2}\right)^2$$

Geometric progression ($r \neq 1$):

$$\sum_{k=0}^{n-1} ar^k = a\frac{r^n - 1}{r-1}$$

For $r = 1$, the sum is an.

Alternating Sum

```
# 1 - 2 + 3 - 4 + ... +/- n
alt_sum = -n // 2 if n % 2 == 0 else (n + 1) // 2

# 1^k - 2^k + 3^k - ... +/- n^k
alt_pow_sum = sum((1 if i & 1 else -1) * i**k
                  for i in range(1, n + 1))
```

Binary Search on Answer

First True / Minimum Valid: FFFFFTTTTT

```
def check(x):
    return ...

l, r = ..., ...
ans = -1
while l <= r:
    m = (l + r) // 2
    if check(m):
        ans = m
        r = m - 1
    else:
        l = m + 1
print(ans)
```

Last True / Maximum Valid: TTTTTFFFFF

```
def check(x):
    return ...

l, r = ..., ...
ans = -1
while l <= r:
    m = (l + r) // 2
    if check(m):
        ans = m
        l = m + 1
    else:
        r = m - 1
print(ans)
```

Requirement: `check(x)` must be monotonic over the search interval.

Number Theory

GCD / LCM

```
from math import gcd
lcm = a // gcd(a, b) * b

# Euclid; useful when b itself is supplied via b % a
# without constructing a huge decimal integer.
def gcd_euclid(a, b):
    while a:
        a, b = b % a, a
    return b
```

Prime Factorization by Trial Division

```
from collections import defaultdict
from math import isqrt

def factors(n):
    f = defaultdict(int)
    while n % 2 == 0:
        f[2] += 1
        n //= 2
    p = 3
    while p * p <= n:
        while n % p == 0:
            f[p] += 1
            n //= p
        p += 2
    if n > 1:
        f[n] += 1
    return f
```

If $n = \prod p_i^{e_i}$:

$$\tau(n) = \prod (e_i + 1), \qquad \sigma(n) = \prod \frac{p_i^{e_i+1} - 1}{p_i - 1}.$$

```
def num_divisors(fact):
    ans = 1
    for e in fact.values():
        ans *= e + 1
    return ans
```

Divisor Count / Sum for Every $1 \ldots N$

```
N = 10**5
divcnt = [0] * (N + 1)
for i in range(1, N + 1):
    for j in range(i, N + 1, i):
```

```
        divcnt[j] += 1

# Sum of divisors instead:
# divsum[j] += i
```

Ordinary Sieve of Eratosthenes

```
def sieve(N):
    is_prime = bytearray(b'\x01') * (N + 1)
    if N >= 0: is_prime[0] = 0
    if N >= 1: is_prime[1] = 0
    for p in range(2, int(N**0.5) + 1):
        if is_prime[p]:
            start = p * p
            is_prime[start:N+1:p] = b'\x00' * (
                (N - start) // p + 1)
    return is_prime
```

Odd-Only Bytearray Sieve

Index i represents the odd number $2i + 1$.

```
from math import isqrt

def sieve_bytearray(N):
    size = (N + 1) // 2
    s = bytearray(b'\x01') * size
    if size:
        s[0] = 0                    # 1 is not prime

    lim = (isqrt(N) - 1) // 2
    for i in range(1, lim + 1):
        if s[i]:
            p = 2 * i + 1
            start = (p * p) // 2
            s[start::p] = b'\x00' * (
                (size - start - 1) // p + 1)
    return s

N = 10**6 + 7
a = sieve_bytearray(N)

def is_prime_fast(x):
    if x == 2:
        return True
    if x < 2 or x % 2 == 0 or x > N:
        return False
    return bool(a[x // 2])
```

Compact Prime List from Odd Sieve

```
from array import array

primes = array('I', [2])
for i in range(1, len(a)):
    if a[i]:
        primes.append(2 * i + 1)
```

Smallest Prime Factor (SPF) — Many Queries

Preprocess once; each factorization takes $O(\log n)$ factors with multiplicity.

```
N = 10**6 + 9
spf = list(range(N + 1))

for i in range(2, int(N**0.5) + 1):
    if spf[i] == i:
        for j in range(i * i, N + 1, i):
            if spf[j] == j:
                spf[j] = i

def factorize(n):
    res = []
    while n > 1:
        p = spf[n]
        res.append(p)
        n //= p
    return res
```

All Divisors of One Number

```
def all_divisors(n):
    d = []
    i = 1
    while i * i <= n:
        if n % i == 0:
            d.append(i)
            if i != n // i:
                d.append(n // i)
        i += 1
    return d              # sort(d) if order matters
```

Goldbach Pair for an Even Integer

```
def goldbach(n, is_prime):
    if n < 4 or n % 2:
        return None
    for p in range(2, n // 2 + 1):
        if is_prime[p] and is_prime[n - p]:
            return p, n - p
    return None
```

Binary Exponentiation

```
def bin_exp(a, b, mod=None):
    res = 1
    while b:
        if b & 1:
            res = res * a if mod is None else res * a % mod
        a = a * a if mod is None else a * a % mod
        b >>= 1
    return res

# Python built-in modular exponentiation:
# pow(a, b, MOD)
```

Deterministic Miller–Rabin for 64-bit Integers

```
def is_prime_mr(n):
    if n < 2:
        return False
    small = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37)
    for p in small:
        if n % p == 0:
            return n == p

    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1

    for a in (2, 325, 9375, 28178, 450775,
              9780504, 1795265022):
        if a % n == 0:
            continue
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1:
                break
        else:
            return False
    return True
```

Factorials, Inverse Factorials & nCr

Requires prime `MOD` and $n <$ MOD for Fermat inverse.

```
MOD = 10**9 + 7
MAXN = 2000

fact = [1] * (MAXN + 1)
for i in range(1, MAXN + 1):
    fact[i] = fact[i - 1] * i % MOD

inv_fact = [1] * (MAXN + 1)
inv_fact[MAXN] = pow(fact[MAXN], MOD - 2, MOD)
for i in range(MAXN, 0, -1):
    inv_fact[i - 1] = inv_fact[i] * i % MOD

def nCr(n, r):
    if r < 0 or r > n:
        return 0
    return fact[n] * inv_fact[r] % MOD * inv_fact[n-r] % MOD
```

Geometry

Triangle Area — Heron's Formula

For sides a, b, c and $s = (a + b + c)/2$:

$$K = \sqrt{s(s-a)(s-b)(s-c)}.$$

Law of Cosines

For triangle ABC, side AB is opposite angle C:

```
from math import acos, degrees
C = degrees(acos((BC*BC + CA*CA - AB*AB)
                 / (2 * BC * CA)))
```

Clamp the cosine to $[-1, 1]$ if floating-point error is possible.

Polygon Area — Shoelace Formula

```
def polygon_area2(points):
    # Returns twice the area; exact for integer coordinates.
    area2 = 0
    n = len(points)
    for i in range(n):
        x1, y1 = points[i]
        x2, y2 = points[(i + 1) % n]
        area2 += x1 * y2 - y1 * x2
    return abs(area2)

# area = polygon_area2(points) / 2
```

Graph Algorithms

Iterative DFS with Traversal Depth

For a tree this is depth from `start`; in a general graph it is DFS-tree depth.

```
def dfs_max_depth(start, graph):
    seen = {start}
    stack = [(start, 1)]
    best = 0
    while stack:
        u, dep = stack.pop()
        best = max(best, dep)
        for v in graph[u]:
            if v not in seen:
                seen.add(v)
                stack.append((v, dep + 1))
    return best
```

Binary Tree Inorder Traversal

```
def inorder(tree, root):
    ans, st = [], []
    cur = root
    while cur is not None or st:
        while cur is not None:
            st.append(cur)
            cur = tree.get(cur, (None, None))[0]
        cur = st.pop()
        ans.append(cur)
        cur = tree.get(cur, (None, None))[1]
    return ans
```

N-ary Tree Preorder & Postorder

```
def dfs_orders(tree, root):
    pre, post = [], []
    st = [(root, 0)]
    while st:
        u, i = st.pop()
        if i == 0:
            pre.append(u)
        ch = tree.get(u, [])
        if i < len(ch):
            st.append((u, i + 1))
            st.append((ch[i], 0))
        else:
            post.append(u)
    return pre, post
```

Dijkstra — Non-negative Edge Weights

Adjacency list: `graph[u] = [(v,w), ...]`.

```
import heapq

def dijkstra(n, graph, src=1):
    INF = float('inf')
    dist = [INF] * (n + 1)
    par = [-1] * (n + 1)
    dist[src] = 0
    pq = [(0, src)]

    while pq:
        du, u = heapq.heappop(pq)
        if du != dist[u]:
            continue
        for v, w in graph[u]:
            nd = du + w
            if nd < dist[v]:
                dist[v] = nd
                par[v] = u
                heapq.heappush(pq, (nd, v))
    return dist, par
```

```
def restore_path(par, src, target):
    path = []
    u = target
    while u != -1:
        path.append(u)
        if u == src:
            return path[::-1]
        u = par[u]
    return []
```

Topological Sort — Kahn's Algorithm

```
from collections import deque

def topo_sort(graph, n):
    indeg = [0] * (n + 1)
    for u in range(1, n + 1):
        for v in graph[u]:
            indeg[v] += 1

    q = deque(i for i in range(1, n + 1)
              if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)

    return order if len(order) == n else []  # [] => cycle
```

Data Structures

DSU / Union–Find

1-indexed, iterative path compression, union by size.

```
n = 10
parent = list(range(n + 1))
sz = [1] * (n + 1)

def find(u):
    root = u
    while parent[root] != root:
        root = parent[root]
    while u != root:
        nxt = parent[u]
        parent[u] = root
        u = nxt
    return root

def union(u, v):
    u, v = find(u), find(v)
    if u == v:
        return False
    if sz[u] < sz[v]:
        u, v = v, u
    parent[v] = u
    sz[u] += sz[v]
    return True

# Same component: find(a) == find(b)
# Components after all unions:
# len({find(i) for i in range(1, n + 1)})
```

Fenwick Tree / Binary Indexed Tree

1-based indexing. Build $O(N)$, point update and prefix/range queries $O(\log N)$, space $O(N)$.

Python Template

```
N = 200005
bit = [0] * (N + 1)
arr = [0] * (N + 1)

# Build from a 0-indexed list, O(N)
def build(values):
    if len(values) > N:
        raise ValueError("Too many values")
    bit[:] = [0] * (N + 1)
    arr[:] = [0] * (N + 1)
    for i, x in enumerate(values, 1):
        arr[i] = x
        bit[i] = x
    for i in range(1, N + 1):
        j = i + (i & -i)
        if j <= N:
            bit[j] += bit[i]

# Add val at position i
# (does not replace it)
def update(i, val):
    arr[i] += val
    while i <= N:
        bit[i] += val
        i += i & -i

# Prefix sum [1, i]
def query(i):
    ans = 0
    while i > 0:
        ans += bit[i]
        i -= i & -i
    return ans

# Inclusive range sum [l, r]
def range_query(l, r):
    return query(r) - query(l - 1)

# Replace arr[i] with val
def set_value(i, val):
    update(i, val - arr[i])

# First index i with query(i) >= k
# Needs non-negative frequencies
def kth(k):
    if k <= 0 or k > query(N):
        return -1
    i = 0
    step = 1 << (N.bit_length() - 1)
    while step:
        nxt = i + step
        if nxt <= N and bit[nxt] < k:
            k -= bit[nxt]
            i = nxt
        step >>= 1
    return i + 1
```

Initialize

```
# Build once from an existing array:
build([2, 4, 1, 5, 3])
# Or start empty and call update(i, x)
```

Recognize These Problem Patterns

1. Dynamic Range Sum
Problem: Given an array, support point additions/assignments and sum queries over $[l, r]$.
Approach: Store values by position. Use `update(i,x)`, `set_value(i,x)`, and `range_query(l,r)`.

2. K-th Smallest in a Multiset
Problem: Insert/delete integer values (including duplicates) and find the k-th smallest.
Approach: Store frequencies at value indices. Insert: `update(x,1)`; delete: `update(x,-1)`; answer: `kth(k)`.

3. K-th Remaining Element
Problem: Given an unsorted array, repeatedly remove and print the k-th still-active element (CSES List Removals).
Approach: Store 1 for active original positions and 0 for removed ones: `build([1]*n)`, `pos=kth(k)`, print `a[pos-1]`, then `update(pos,-1)`.

4. Count Values in a Range
Problem: Maintain a multiset and count how many values lie in $[l, r]$.
Approach: Store frequencies by value and use `range_query(l,r)` to count occurrences.

Quick Usage Examples

```
# Normal range sum:
build([2, 4, 1, 5, 3])
range_query(2, 4)  # 10
update(3, 5)       # add 5
range_query(2, 4)  # 15
set_value(2, 10)   # replace index 2

# CSES List Removals (fresh tree):
a = [2, 6, 1, 4, 2]
removals = [3, 1, 3, 1, 1]
build([1] * len(a))
ans = []
for k in removals:
    pos = kth(k)
    ans.append(a[pos - 1])
    update(pos, -1)
# ans: [1, 2, 2, 6, 4]
```

SUM → XOR Customization

Replace the following operations; the loops stay unchanged.

Function	Change
`build`	`bit[j] ^= bit[i]`
`update`	`arr[i] ^= val`, `bit[i] ^= val`
`query`	`ans ^= bit[i]`
`range_query`	`query(r) ^ query(l-1)`
`set_value`	`update(i, val ^ arr[i])`

Note: `kth(k)` is not supported in XOR mode.

Important Notes

- Indices: $1 \le i \le N$. Never call `update(0,x)`.
- `update(i,x)` adds; `set_value(i,x)` replaces.
- `kth(k)` finds the first index whose prefix sum is $\ge k$. Values must be non-negative.
- For frequency queries, never delete an absent value. Compress large/negative value coordinates.
- Reset the tree between independent examples.